Department of Computer Science and Engineering
Premier University

CSE 481: Contemporary Course of Computer Science

Assignment: Intelligent Smart Transportation & Traffic Management System

Submitted by:

Name	Shuvra Roy
ID	02222100051011093
Section	C
Session	Fall 2025
Semester	8th Semester
Submission Date	17.05.2026

Submitted to:
Wong May Nu
Lecturer, Department of CSE
Premier University, Chittagong

Remarks

1 Executive Summary

Traffic congestion is a major problem in big cities of Bangladesh such as Dhaka, Chittagong, Sylhet, Khulna, and Rajshahi. Due to heavy traffic, road accidents, and delayed emergency services, the country loses a huge amount of time and money every year. To solve these problems, this project proposes an *Intelligent Smart Transportation and Traffic Management System (ISTMS)*.

The proposed system uses IoT devices, AI/ML models, edge computing, and cloud services to improve traffic management. The system can monitor real-time traffic conditions from different intersections and predict future traffic congestion using machine learning models.

Main features of the proposed system are:

- Real-time traffic monitoring at more than 500 intersections
- Traffic prediction 15–30 minutes earlier using AI models
- Automatic traffic signal control to reduce waiting time
- Accident detection and emergency vehicle support
- Cloud-based system using AWS services

The system is expected to reduce traffic congestion and improve emergency response time. The estimated monthly AWS operational cost of the system is around **$18,450 per month**.

2 Investigation: Limitations of Current Traffic Systems in Bangladesh

2.1 Current State Analysis

Bangladesh's traffic management infrastructure is largely outdated and reactive:

- **Manual Signal Control:** Most signals in Dhaka are manually operated by traffic police, creating inconsistent timing and human error.
- **No Predictive Capability:** There is no mechanism to anticipate congestion before it occurs.
- **Poor Emergency Response:** Ambulances and fire trucks are often trapped in gridlock for 20–45 minutes due to no priority corridor system.
- **Limited Sensor Infrastructure:** Only a handful of intersections have any form of sensor-based monitoring.
- **Data Silos:** Traffic data, accident records, and public transport GPS data are maintained by different agencies and are not integrated.
- **Absence of Adaptive Control:** Signal timings are fixed pre-sets, unable to respond dynamically to real-time demand.

2.2 Root Cause Analysis

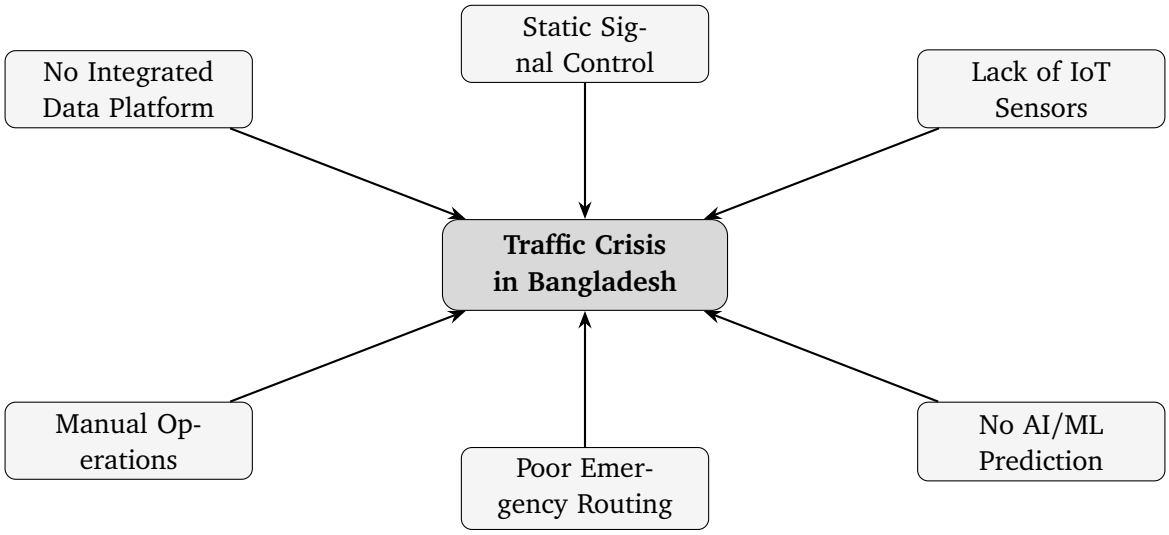

Figure 1: Root Causes of Traffic Crisis in Bangladesh

3 Proposed System Architecture

3.1 Three-Tier Hybrid Architecture Overview

The ISTMS is built on a three-tier model: the **IoT/Perception Layer**, the **Edge Computing Layer**, and the **Cloud/Intelligence Layer**. This separation resolves the core tension between low-latency local decisions and high-accuracy centralised analytics.

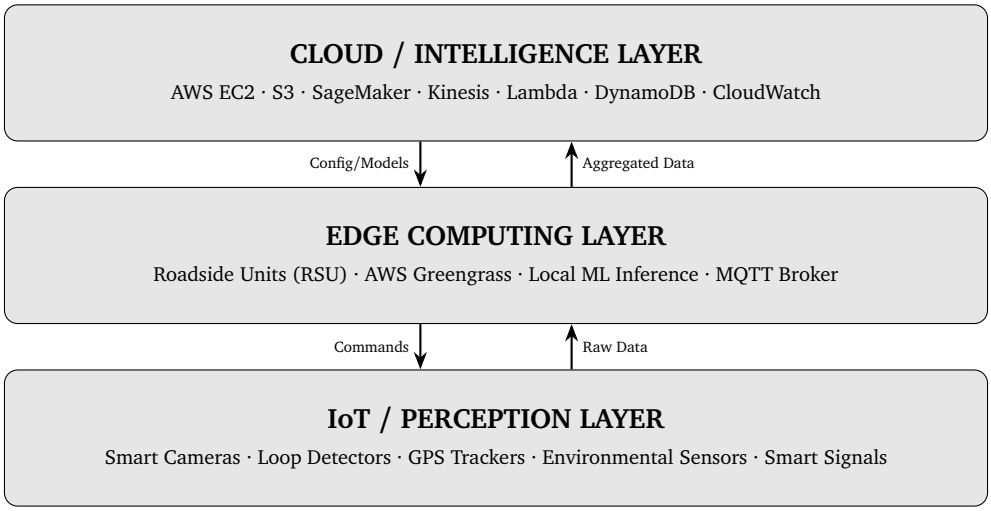

Figure 2: Three-Tier Hybrid Architecture of ISTMS

3.2 Layer 1 — IoT / Perception Layer

3.2.1 IoT Devices and Sensors

Device	Function	Specification / Protocol
Smart CCTV Cameras	Vehicle counting, traffic monitoring, and license plate detection	4K camera, 30 fps, H.265 compression, supports RTSP and ONVIF
Inductive Loop Detectors	Counts vehicles and measures traffic flow at intersections	Installed under road surface, works continuously, RS-485 output
Radar Speed Sensors	Measures vehicle speed and detects traffic violations	K-band radar, accuracy around ±2 km/h, supports Modbus RTU
Smart Traffic Signal Controllers	Controls traffic lights automatically based on traffic condition	Supports NTCIP 1202 standard, dual-ring controller, PoE support
GPS-Enabled Bus/Vehicle Trackers	Tracks the real-time location of buses and vehicles	GNSS support, updates every 10 seconds, uses 4G LTE network
Environmental Sensors	Monitors air quality, weather, and road condition	Can detect PM2.5, NO2, CO, temperature and rain, supports I2C/SPI
Emergency Vehicle Transponders	Gives priority to emergency vehicles on roads	IEEE 802.11p DSRC communication, 300m range, low latency
Pedestrian Push-Button Sensors	Used for pedestrian crossing requests	Accessible tactile button with ZigBee communication

3.2.2 Data Volume Estimation

Each intersection generates approximately:

$$
\begin{aligned}
D_{\text{camera}} &= 4 \text{ cameras} \times 2\,\text{Mbps (compressed)} = 8\,\text{Mbps/intersection} \\
D_{\text{sensors}} &\approx 50\,\text{KB/min per intersection (all sensors)} \\
D_{\text{total}} &\approx 8\,\text{Mbps} \times 500 \text{ intersections} = 4\,\text{Gbps peak}
\end{aligned}
$$

Edge pre-processing reduces cloud-bound data to $\approx 5\%$ of raw volume, yielding $\sim$200 Mbps to cloud.

3.3 Layer 2 — Edge Computing Layer

3.3.1 Roadside Unit (RSU) Design

Each RSU covers 3–5 intersections and performs:

- Real-time vehicle counting, speed calculation, and density estimation
- Local ML inference for signal timing optimisation (latency $<$100 ms)

- MQTT broker for device communication
- Anomaly detection (accident/incident) with immediate local alert
- Data compression and aggregation before cloud upload

Recommended Hardware per RSU:

Component	Specification
Processor	NVIDIA Jetson AGX Orin (275 TOPS) or similar processor
RAM	32 GB LPDDR5
Storage	1 TB NVMe SSD
Network	4G LTE, Wi-Fi 6, and Fibre uplink support
Power	Solar power with UPS backup for 72 hours
OS	Ubuntu 22.04 LTS with AWS Greengrass v2

3.3.2 AWS Greengrass Deployment

AWS IoT Greengrass v2 enables:

- Secure OTA updates of edge ML models from AWS SageMaker
- Local Lambda execution for signal control logic
- Offline operation — RSUs continue functioning during cloud outages
- Synchronisation of buffered data upon connectivity restoration

3.4 Layer 3 — Cloud / Intelligence Layer

3.4.1 AWS Services Architecture

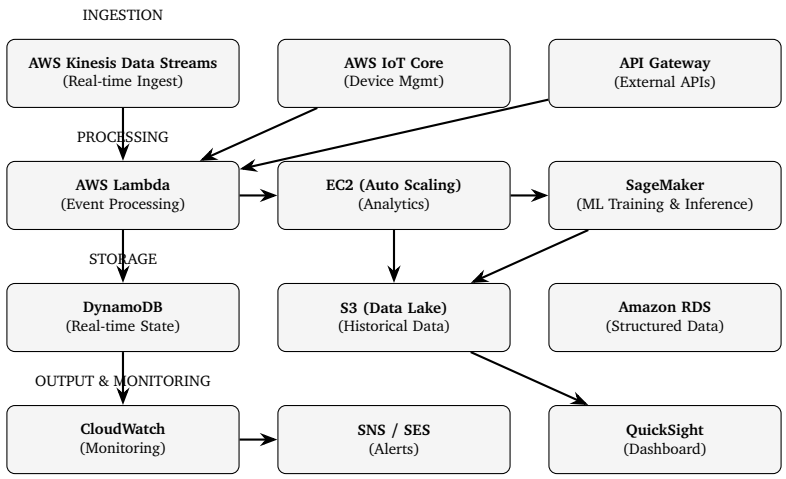

Figure 3: AWS Services Architecture for ISTMS

4 AI Models for Traffic Management

4.1 Traffic Density & Congestion Prediction

4.1.1 Model: Spatio-Temporal Graph Convolutional Network (ST-GCN)

In this system, traffic intersections are represented as a graph $G = (V, E)$. Here, each node represents an intersection and each edge represents a road connection between two intersections.

Input Features:

For each intersection, the model takes some traffic-related information as input:

$$\mathbf{x}_i^{(t)} = \left[\text{density}_i^{(t)}, \text{avg\_speed}_i^{(t)}, \text{occupancy}_i^{(t)}, \text{time\_of\_day}, \text{day\_of\_week}\right]$$

These features help the model understand the current traffic condition.

Graph Convolution:

The graph convolution layer is used to learn the relationship between connected intersections.

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right)$$

Here, $\tilde{A} = A + I$ is the adjacency matrix with self-connections, $\tilde{D}$ is the degree matrix, and σ represents the ReLU activation function.

Temporal Module:

A GRU (Gated Recurrent Unit) layer is used to process traffic data over time.

$$h_t = \text{GRU}(h_{t-1},\ H_t^{(\text{GCN})})$$

This helps the model learn traffic patterns from previous time steps.

Output:

The model predicts the future congestion level $\hat{y}_i^{(t+k)}$ for each intersection. Predictions are generated for future time steps such as 5 minutes, 15 minutes, and 30 minutes ahead.

Training Details:

- Historical traffic data from at least 6 months is used
- MAPE (Mean Absolute Percentage Error) is used as the loss function
- The model is trained on AWS SageMaker using `ml.p3.2xlarge` instances
- The system is re-trained weekly and a full re-training is done every month

4.2 Adaptive Traffic Signal Control

4.2.1 Model: Deep Q-Network (DQN) Reinforcement Learning

Signal control is framed as a Markov Decision Process (MDP):

Component	Definition
State s_t	Vehicle queue length, waiting time, signal phase duration, and time of day
Action a_t	Select the next traffic signal phase such as NS-Green, EW-Green, or Pedestrian crossing
Reward r_t	$r_t = -\sum_i (\text{queue}_i + \alpha \cdot \text{wait}_i)$. This reward function helps reduce total vehicle waiting time and traffic congestion.
Q-function	$Q(s,a) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a\right]$. It estimates the future reward for taking action a in state s.

Training: Simulated using SUMO (Simulation of Urban MObility) with Dhaka road network. Deployed at edge RSUs for $<$50 ms signal switching decisions.

4.3 Accident & Anomaly Detection

4.3.1 Model: YOLOv8 + LSTM Hybrid

In this system, a hybrid model using YOLOv8 and LSTM is used for accident and anomaly detection.

- **YOLOv8**: This model is used for real-time object detection. It can detect vehicle accidents, wrong-way driving, and pedestrians entering restricted areas from traffic camera video frames.
- **LSTM-based Anomaly Detection**: The LSTM model monitors traffic data such as vehicle speed and traffic density over time. If there is a sudden drop in speed or abnormal traffic pattern, the system considers it as a possible accident.

The alert decision is calculated using the following condition:

$$\text{Alert} = \begin{cases} \text{TRUE} & \text{if YOLO\_conf} > 0.85 \text{ or LSTM\_anomaly} > \theta \\ \text{FALSE} & \text{otherwise} \end{cases}$$

If an accident or anomaly is detected, the system sends an SNS notification to emergency services. Then the shortest emergency route is calculated using Dijkstra's algorithm within a few seconds.

4.4 Route Optimisation

The system uses a modified Dijkstra's algorithm to find the shortest and fastest route for emergency vehicles.

$$w(u,v)_t = \frac{\text{distance}(u,v)}{\text{free\_flow\_speed}(u,v)} \times (1 + \text{congestion\_index}^t_{(u,v)})$$

Here, the route weight changes dynamically based on current traffic congestion. The system updates the route every 30 seconds and sends the updated route information to vehicle navigation systems and traffic management dashboards.

5 Cloud Infrastructure Design

5.1 Deployment Model: Hybrid Cloud

5.1.1 Why Hybrid Cloud is Used

A Hybrid Cloud model is used in this system because it gives better flexibility, security, and performance.

- Edge devices (RSUs) can make fast real-time decisions locally with very low delay.
- AWS Public Cloud is used for data analytics, machine learning training, long-term data storage, and dashboard services.
- Private servers at Traffic Management Centres (TMCs) are used to store sensitive traffic and law-enforcement data securely.
- Another advantage is that the edge system can continue working even if the internet or WAN connection fails.

5.2 Service Model

Layer	**Service Model**	**Usage**
Infrastructure	IaaS	EC2 instances for analytics, Kinesis streams, and S3 storage
Platform	PaaS	AWS SageMaker for machine learning, AWS IoT Core, and Lambda for serverless services
Application	SaaS	Dashboard using QuickSight and alert system using SNS

5.3 Compute, Storage, and Networking

5.3.1 Compute

Purpose	**Instance Type**	**Count**	**vCPU / RAM**
Analytics / Stream Processing	`m6i.4xlarge`	4 (Auto-scaled)	16 / 64 GB
ML Inference API	`c6i.2xlarge`	3	8 / 16 GB
ML Training (Scheduled)	`p3.2xlarge`	2	8 / 61 GB + GPU
Database (RDS PostgreSQL)	`db.r6g.large`	2 (Multi-AZ)	2 / 16 GB
Bastion / VPN	`t3.micro`	1	2 / 1 GB

5.3.2 Storage

Service	Purpose	Volume
Amazon S3 (Standard)	Stores raw sensor data and video clips	50 TB/month
Amazon S3 (Glacier)	Stores archived data older than 90 days	200 TB cumulative
Amazon DynamoDB	Stores real-time intersection state data	500 GB
Amazon RDS (PostgreSQL)	Stores incident records and reports	2 TB
EBS Volumes	Storage for EC2 instances	2 TB total

5.3.3 Networking

- **AWS VPC** with public and private subnets across 2 Availability Zones
- **AWS Direct Connect** / Site-to-Site VPN from Traffic Management Centres
- **Application Load Balancer** for the inference API
- **CloudFront CDN** for the national dashboard
- **Data transfer:** ∼200 Mbps inbound from edge nodes

5.4 Security and Compliance

Security Domain	Implementation
Data in Transit	TLS 1.3 is used for all API and MQTT communication
Data at Rest	AES-256 encryption is used for S3, DynamoDB, and RDS storage
Access Control	AWS IAM roles with least-privilege access policies
Network Security	VPC Security Groups, NACLs, and WAF for public endpoints
Device Auth	X.509 certificates are used through AWS IoT Core
Audit Logging	AWS CloudTrail and VPC Flow Logs are used for monitoring and logging
Compliance	Follows Bangladesh Digital Security Act 2018 and international ITS standards

6 AWS Cost Estimation

6.1 Monthly Cost Breakdown (AWS Pricing Calculator)

All prices in USD, **AWS Asia Pacific (Singapore) region** — closest to Bangladesh.

Service Category	Detail	Monthly Cost (USD)
Compute		
EC2 Analytics	4× m6i.4xlarge On-Demand (730 hrs)	$2,189
EC2 Inference API	3× c6i.2xlarge On-Demand (730 hrs)	$825
EC2 ML Training	2× p3.2xlarge On-Demand (200 hrs/-month)	$620
EC2 RDS	2× db.r6g.large Multi-AZ	$380
Lambda	50M invocations with 512 MB average memory	$110
Storage		
S3 Standard	50 TB storage	$1,150
S3 Glacier	200 TB archival storage	$820
DynamoDB	500 GB database with 500K WCU/RCU per day	$740
RDS Storage	2 TB gp3 SSD storage	$230
EBS	2 TB gp3 volume	$160
AI / ML		
SageMaker Training	ml.p3.2xlarge instance, 200 hrs/month	$980
SageMaker Inference	3× ml.c6i.xlarge endpoints	$650
Data Streaming & IoT		
Kinesis Data Streams	200 shards with 200 Mbps streaming	$1,440
IoT Core	500 devices with 5M messages/day	$750
IoT Greengrass	100 RSU devices	$350
Networking		
Data Transfer Out	10 TB/month data transfer to dashboards	$920
VPN / Direct Connect	Site-to-Site VPN for 5 TMCs	$230
CloudFront CDN	Dashboard content delivery	$180
Load Balancer (ALB)	Used for inference API	$85
Analytics & Monitoring		
Kinesis Firehose	S3 delivery with 200 GB/day	$420

(continued from previous page)

Service Category	**Detail**	**Monthly Cost (USD)**
CloudWatch	Logs, metrics, and alarms	$310
QuickSight	20 authors and 200 readers	$480
SNS / SES	10M notifications/month	$85
Security		
WAF	10M requests/month	$103
CloudTrail	Management and data event logging	$85
AWS Shield Standard	Included without extra cost	$0
	TOTAL ESTIMATED MONTHLY COST	**$18,452**

6.2 Cost Optimisation Strategies

- **Reserved Instances (1-year):** Purchasing RIs for always-on EC2 (analytics, RDS) saves ∼40%, reducing EC2 costs by ∼$1,200/month.
- **Spot Instances for Training:** ML training jobs run on Spot, saving ∼70% (≈$430/month saved).
- **S3 Intelligent-Tiering:** Auto-moves infrequently accessed data to cheaper tiers.
- **Edge Preprocessing:** Compressing and aggregating at RSU reduces data transfer and Kinesis shard costs significantly.
- **Estimated optimised cost:** ≈ **$14,800/month** with RIs and Spot.

7 Evaluation and Design Justification

7.1 Conflicting Technical Requirements and Resolutions

Conflict	Tension	Resolution
Low Latency vs. Centralised Processing	Edge processing increases system complexity, while cloud processing provides more computing power	Hybrid approach: edge is used for real-time processing (<100 ms) and cloud is used for learning and analytics (>1 minute)
Data Sharing vs. User Privacy	Vehicle and fleet tracking can affect commuter privacy	GPS data is anonymised at the edge, and only aggregated traffic data is sent to the cloud with IAM and encryption security
High Accuracy vs. Computational Cost	Deep learning models with high accuracy need high computational resources	Lightweight quantised models are used at the edge, while full models run in the cloud with TensorRT optimisation
Scalability vs. Cost	Expanding the system to more cities increases operational cost	Auto-scaling groups, serverless Lambda, and S3 lifecycle policies are used to reduce cost

7.2 Adherence to Standards

- **Communication Protocols:** MQTT 3.1.1 (IoT-to-edge), HTTP/2 REST (edge-to-cloud), WebSocket (dashboard real-time updates)
- **ITS Standards:** NTCIP 1201/1202 (traffic signal control), SAE J2735 (V2X messaging), ISO 21217 (ITS architecture)
- **Security:** TLS 1.3 everywhere, AES-256 at rest, AWS IAM least-privilege, OWASP API security guidelines
- **Data Format:** JSON/Protobuf for sensor messages, Parquet for analytics storage

7.3 Infrequent Challenges and Mitigations

Challenge	Mitigation Strategy
Sensor Failures	Backup sensors are used at important intersections. Watchdog timers, auto-calibration algorithms, and anomaly detection are used when data is missing for more than 60 seconds.
Network Instability	RSUs can store data locally for up to 72 hours. 4G LTE is used as the main network and licensed narrowband is used as a backup. Kinesis manages delayed or unordered data delivery.
Large-Scale Streaming	Kinesis automatically scales shards, Firehose sends batched data to S3, Lambda fan-out is used, and DynamoDB adaptive capacity helps manage large traffic loads.
Power Outages	Solar panels and 72-hour UPS backup are used for each RSU. If power fails, the system switches to preset traffic signal timings.
Model Drift	SageMaker Model Monitor checks for data distribution changes. If drift is detected, an automatic retraining pipeline is started.

8 Public Safety and Emergency Handling

8.1 Emergency Response Workflow

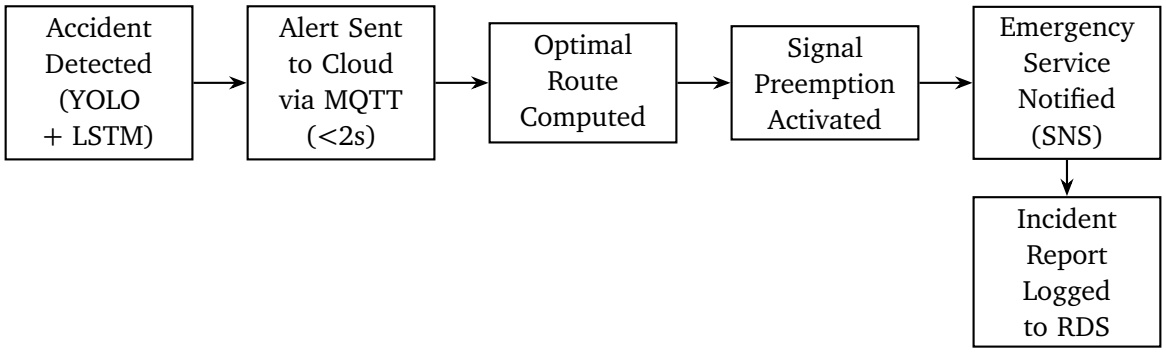

Figure 4: Emergency Response Workflow

8.1.1 Performance Target

The main target of the system is to reduce emergency response delay. After detecting an accident, the system should create a green corridor for emergency vehicles within **30 seconds**. Currently, in Dhaka, this process usually takes around **8–15 minutes**.

9 Scalability and Fault Tolerance

9.1 Multi-City Scalability

- **Multi-tenancy:** Each city is a separate *tenant* with isolated VPC subnets, DynamoDB table namespaces, and S3 prefixes.
- **Horizontal Scaling:** EC2 Auto Scaling Groups expand from 4 to 20 instances during peak hours; Kinesis shards scale with data volume.
- **SageMaker Multi-Model Endpoints:** One endpoint hosts per-city traffic models, reducing inference infrastructure cost.

9.2 High Availability

Component	HA Strategy
EC2 Analytics	Uses Multi-AZ Auto Scaling Group with ALB health checks for high availability
RDS	Uses Multi-AZ deployment with automatic failover in less than 1 minute
DynamoDB	Globally distributed service with 99.999% SLA availability
Kinesis	Uses 3 Availability Zone replication and keeps data for 7 days
RSU Edge	Can work independently and stores data locally if the cloud connection is unavailable

Target SLA: 99.95% uptime for cloud services; 99.9% for edge RSUs (accounting for power/network).

10 System Performance Metrics

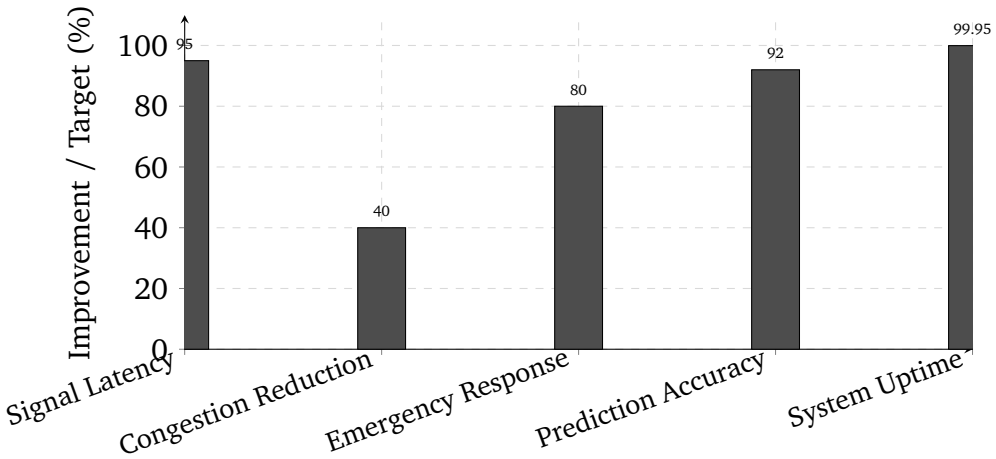

Figure 5: Expected System Performance Targets

Metric	Baseline (Current)	Target (ISTMS)
Signal adjustment latency	Manual process (takes minutes)	Less than 100 ms using edge processing
Average congestion reduction	Not available	40% reduction target
Emergency response time	8–15 minutes	Preemption starts within 30 seconds
Traffic prediction accuracy (MAPE)	N/A	Less than 8%
System uptime	N/A	99.95% availability
Accident detection time	Manual reporting	Less than 5 seconds

11 Stakeholder Analysis

Stakeholder	Primary Need	How ISTMS Addresses It
Government Agencies	Policy compliance and cost control	Provides dashboard reporting, cost-optimized AWS architecture, and open data APIs
Commuters	Reduced travel time and better reliability	Helps reduce congestion by 40% and supports real-time route guidance integration
Traffic Police	Automated support and incident alerts	Uses automated anomaly detection and reduces the need for manual signal control
Emergency Responders	Fast and clear traffic corridors	Supports preemption activation within 30 seconds and dynamic route optimization
Public Transport Operators	Predictable transport schedules	Uses GPS integration and adaptive signal priority for buses

12 Conclusion

The proposed **Intelligent Smart Transportation and Traffic Management System (ISTMS)** provides a smart and practical solution for traffic problems in Bangladesh. The system uses IoT devices, edge computing, AI/ML models, and AWS cloud services to improve traffic management and emergency response.
Main technologies used in this system are:

- IoT devices such as cameras, loop detectors, and GPS trackers
- Edge computing for fast real-time traffic control

- AI/ML models for traffic prediction, signal control, and accident detection
- AWS cloud services for storage, analytics, and system management

The proposed system can help reduce traffic congestion, improve emergency vehicle movement, and provide reliable traffic monitoring. The estimated congestion reduction is around **40%**, emergency support response can start within **30 seconds**, and the system availability target is **99.95%**.
The hybrid cloud architecture also helps the system continue working even if network problems occur. This system can first be implemented in Dhaka and later expanded to other major cities in Bangladesh.

Premier University
Department of Computer Science and Engineering

CSE 481: Contemporary Course of Computer Science

Title: Intelligent Smart Transportation and Traffic Management System

Submitted by:

Name	Arnab Shikder
Student ID:	0222210005101098
Section:	C
Semester:	8th
Session:	Fall 2025
Submission Date:	17.05.2026

Submitted to:
Wong May Nu
Lecturer, Department of CSE
Premier University, Chattogram

Remarks:

1 Introduction

Bangladesh is one of the world's most densely populated countries, and its major cities—especially Dhaka and Chattogram—suffer from severe traffic congestion. Average speeds in Dhaka fall below 7 km/h during peak hours, emergency response times often exceed 16 minutes, and the lack of centralized traffic data limits effective governance.

This report proposes an Intelligent Smart Transportation and Traffic Management System (ISTTMS) to address these challenges. The system combines IoT-based sensors, edge computing for low-latency processing, AI-driven predictive analytics, and AWS cloud infrastructure for scalable processing and long-term optimization.

The proposed architecture supports real-time traffic monitoring, adaptive signal control, anomaly detection, emergency response automation, and a cost-efficient hybrid cloud deployment model. The following sections discuss existing limitations, system architecture, AI model design, cloud and edge infrastructure, security measures, scalability, and AWS-based monthly cost estimation.

2 Inspection

Bangladesh's metropolitan areas operate under a predominantly static, manually controlled traffic management infrastructure. The core limitations fall along four dimensions.

Infrastructure Deficiencies

Traffic signals in Bangladesh are largely timer-based and operate on fixed-cycle logic regardless of actual traffic volume. No feedback mechanism exists between signal controllers and ground-level density. Physical traffic police deployment compensates partially but introduces human variability, fatigue, and inconsistency, particularly during peak hours, adverse weather, or public holidays.

Absence of Predictive and Adaptive Capability

Current systems cannot anticipate congestion build-up. There is no integration with GPS-based vehicle data, no camera-based density estimation, and no machine learning pipeline to forecast bottlenecks before they develop. Congestion is managed reactively rather than proactively, resulting in the critically low average speeds documented above.

Delayed Emergency Response

There is no automated mechanism through which traffic signals are notified of an approaching emergency vehicle. Response time is degraded when ambulances and fire engines are trapped in undifferentiated traffic queues. WHO guidelines recommend a maximum of 8 minutes for life-threatening incidents; Dhaka averages 16–20 minutes.

Data Vacuum

No centralised repository of traffic data exists. This prevents long-term infrastructure planning, route optimisation for public transport, or evidence-based policy decisions. Without structured data collection, patterns in peak-hour behaviour, accident-prone zones, and seasonal variation remain entirely unquantified.

3 Proposed System Architecture

The proposed system is a three-tier hybrid architecture: an IoT and sensing layer at the road level, an edge computing layer at the district level, and a centralised cloud layer for analytics, model training, and governance.

High-Level Architecture Diagram

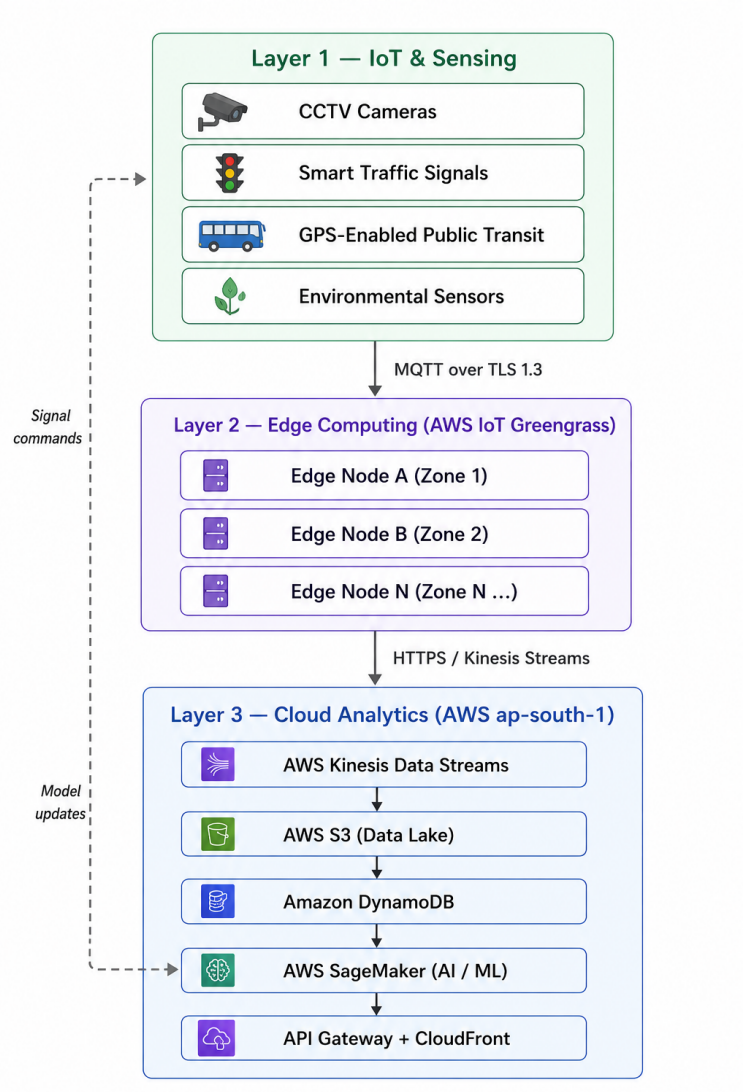

Figure 1: Three-tier hybrid architecture for smart traffic management. Layer 1 contains IoT sensing devices, Layer 2 performs edge processing using AWS IoT Greengrass, and Layer 3 handles cloud analytics and AI/ML services in AWS ap-south-1. The dashed feedback path represents model updates and signal commands.

Layer-by-Layer Description

Layer 1 — IoT and Sensing

- **CCTV Cameras:** High-definition IP cameras at every major intersection. Video frames are processed locally using YOLOv8-nano to count vehicles and classify types before transmission.
- **Smart Traffic Signals:** ESP32-class microcontrollers accept timing commands from the edge node and report current phase state over MQTT with TLS 1.3 encryption.
- **GPS-Enabled Public Transport:** Buses and CNG auto-rickshaws fitted with GPS modules emit location pings every 10 seconds, enabling real-time transit tracking and route adherence monitoring.
- **Environmental Sensors:** Air quality (PM2.5, NO_2), temperature, humidity, and rainfall sensors. Adverse weather inputs dynamically modify the congestion prediction model's thresholds.

Layer 2 — Edge Computing

Each zone (approximately one thana or upazila) is served by a ruggedised edge server running AWS IoT Greengrass v2. Responsibilities include:

- Real-time vehicle density aggregation from cameras at sub-200 ms latency.
- Local inference using a pre-trained ONNX-format signal optimisation model.
- Immediate signal phase adjustment without any cloud round-trip.
- Anomaly detection for accidents, stalled vehicles, and wrong-way drivers, with automated alerts dispatched via a priority MQTT topic.
- Store-and-forward data buffering during upstream network disruption.

Layer 3 — Cloud (AWS)

- **AWS Kinesis Data Streams:** Ingests raw and aggregated telemetry from all edge nodes in real time.
- **AWS S3:** Stores sensor logs, model artefacts, and incident video clips. Lifecycle policies tier data older than 30 days to S3 Glacier.
- **Amazon DynamoDB:** Low-latency key-value store for signal states, active incidents, and vehicle tracking tables.
- **AWS Lambda:** Serverless processors triggered by Kinesis events for lightweight aggregation and routing logic.
- **AWS SageMaker:** Centralised training for traffic forecasting (LSTM), congestion classification (XGBoost), and route optimisation models. Updated models are pushed to edge nodes nightly via Greengrass deployments.

- **Amazon API Gateway + CloudFront:** Serves dashboards to traffic control centres, mobile apps for commuters, and webhooks for emergency services.

4 AI Models: Traffic Prediction, Anomaly Detection, and Route Optimisation

Traffic Flow Forecasting: LSTM Model

The system uses an LSTM-based deep learning model to predict future traffic density at different intersections. Historical traffic data collected from cameras and sensors are combined with external factors such as time of day, day of the week, public holidays, and weather conditions to improve prediction accuracy. The model continuously learns from recent traffic patterns using a rolling 90-day dataset and is retrained weekly through AWS SageMaker Pipelines to maintain reliable forecasting performance.

Traffic Signal Optimisation: Adaptive Signal Control

Traffic signal timing is dynamically adjusted according to real-time vehicle flow at each intersection. The edge computing node processes vehicle counts collected from camera feeds every 30 seconds and updates signal durations automatically. This adaptive approach helps reduce waiting time, improves traffic movement during peak hours, and minimizes congestion without depending on constant cloud connectivity.

Anomaly Detection: Isolation Forest

The system applies an Isolation Forest machine learning model to detect unusual traffic situations such as accidents, sudden congestion, or abnormal vehicle behavior. The model is trained using normal traffic flow patterns and continuously analyzes incoming sensor data to identify anomalies in real time. When suspicious activity is detected, the system automatically generates alerts and triggers emergency response procedures.

Route Optimisation: Dynamic Path Planning

Emergency vehicle routing is handled through a dynamic path optimization mechanism based on real-time road conditions. The routing engine considers current traffic density, predicted congestion levels, and road distance while selecting the most efficient path. Road information is updated regularly, allowing the system to guide emergency vehicles through less congested routes and reduce response time significantly.

Projected Congestion Reduction: Before vs. After

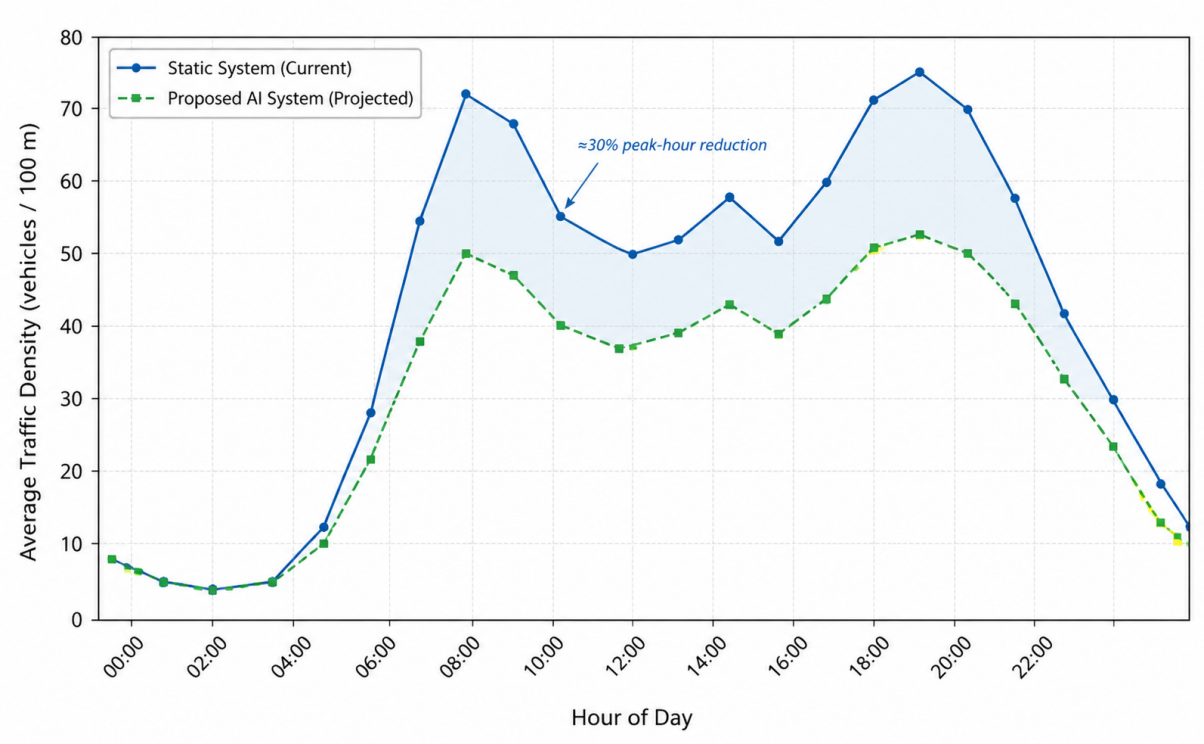

Figure 2: Projected traffic density throughout the day, comparing the current static traffic system with the proposed AI-based adaptive system. The shaded region highlights the estimated congestion reduction achieved during peak hours.

5 Cloud Infrastructure Design

Deployment Model

A **Hybrid Cloud** model is selected. Edge nodes run on-premise via AWS Outposts to guarantee sub-200 ms local decision latency, while centralised training, analytics, and long-term storage reside on the AWS public cloud. This satisfies both the latency requirements of real-time signal control and the elasticity requirements of city-scale machine learning.

Service Model

- **IaaS:** AWS EC2 instances for dedicated inference endpoints and custom networking (VPC, Transit Gateway, Direct Connect to edge nodes).
- **PaaS:** AWS SageMaker (managed ML pipelines), AWS Kinesis (managed streaming), AWS Lambda (event-driven serverless compute).
- **SaaS:** Amazon QuickSight for traffic authority dashboards; Amazon SNS for stakeholder and emergency notifications.

Compute Instances

Table 1: Selected AWS compute instances and their roles.

Component	Instance Type	Count	Purpose
Kinesis Consumer	`c6i.2xlarge`	4	Real-time stream processing
SageMaker Training	`ml.p3.2xlarge`	2	LSTM / XGBoost model training
SageMaker Inference	`ml.m5.xlarge`	4	Online prediction endpoints
API Gateway Backend	`t3.medium`	3	REST API and WebSocket server
DynamoDB (on-demand)	—	—	Auto-scaled, serverless
Lambda Functions	—	—	Serverless event handlers

Storage Architecture

- **AWS S3 (Standard):** Raw sensor logs, model artefacts, and incident video clips. Lifecycle policy moves data older than 30 days to S3 Glacier.
- **Amazon DynamoDB:** Real-time vehicle tracking, signal state tables, and active alert records. On-demand capacity mode handles burst traffic.
- **Amazon Timestream:** Time-series sensor readings for high-speed temporal queries consumed by the operations dashboard.

6 Edge Computing Design

Roadside Unit Hardware Specification

Each RSU is a ruggedised industrial PC deployed in a weather-proof cabinet at major intersections.

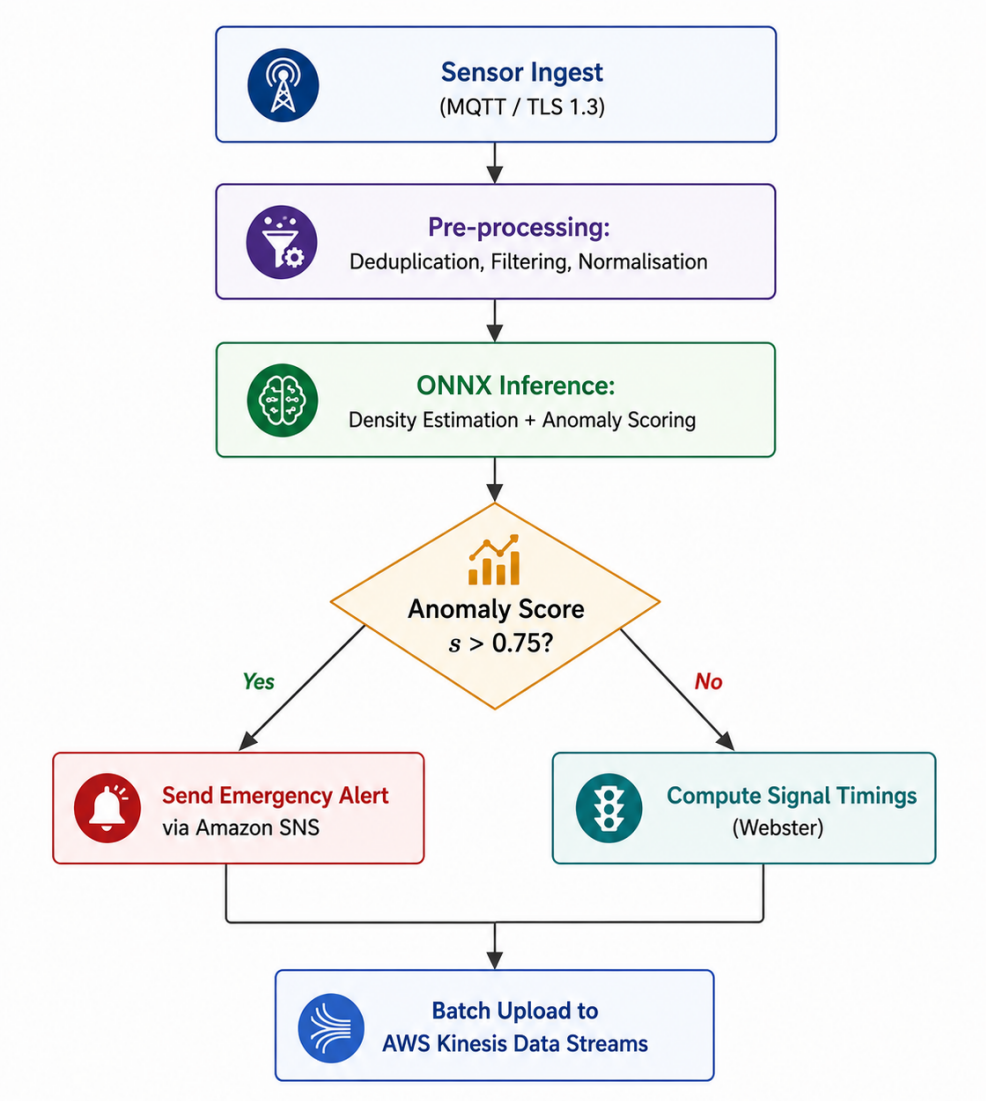

Figure 3: Vertical edge processing pipeline at each Roadside Unit (RSU). Both the emergency alert branch and the signal timing branch converge before batch upload to the cloud through AWS Kinesis Data Streams.

Handling Infrequent Challenges

- **Sensor failure:** Each camera feed includes a 60-second heartbeat. On timeout, the RSU falls back to last-known density values and triggers a maintenance alert. Adjacent RSU readings supplement the missing zone.
- **Network instability:** Store-and-forward buffering holds up to 48 hours of data on local NVMe, compressed with LZ4, and uploaded in priority batches upon connectivity restoration.
- **Large-scale streaming:** Kinesis Data Streams auto-scales shards at 10 MB/s increments; each shard handles 1 MB/s ingestion independently.

7 Security and Compliance

Data Encryption

All data in transit is encrypted using TLS 1.3. Data at rest in S3 and DynamoDB uses AES-256 via AWS Key Management Service (SSE-KMS). Video frames are anonymised at the edge before any cloud upload: faces are blurred using an OpenCV Gaussian mask applied to MTCNN-detected face bounding boxes.

Access Control

AWS IAM enforces least-privilege access across all services. Edge nodes authenticate to AWS IoT Core using X.509 device certificates provisioned via AWS IoT Fleet Provisioning. Role-based access controls separate traffic police, government dashboard users, and emergency service operators into distinct IAM roles with separate permission boundaries.

Communication Protocols

- **IoT to Edge:** MQTT 5.0 over TLS 1.3 (port 8883)
- **Edge to Cloud:** HTTPS REST and AWS IoT Greengrass streams
- **Cloud to Dashboards:** WebSocket (API Gateway) and HTTPS REST
- **Emergency Alerts:** Amazon SNS (SMS, push notification, email)

Compliance

The system adheres to Bangladesh's ICT Act 2006 (amended 2013) and follows the OWASP IoT Security Top 10 for device hardening. ITS communication standards including ISO 14906 (EFC interface) and ETSI ITS-G5 are applied where relevant.

8 Scalability and Fault Tolerance

Multi-City Scalability

The architecture uses an AWS multi-region design with a primary region in `ap-south-1` (Mumbai) and a disaster recovery region in `ap-southeast-1` (Singapore). Each city's RSU network maps to an independent AWS IoT Thing Group, allowing isolated management and billing. New cities are onboarded by provisioning a new Thing Group, deploying pre-built RSU AMI images, and registering with the central Kinesis pipeline. Estimated onboarding time per additional city: 72 hours.

High Availability Design

- All API-facing EC2 instances are deployed across three Availability Zones behind an Application Load Balancer.
- DynamoDB uses global table replication for cross-region redundancy.
- SageMaker inference endpoints use auto-scaling with a minimum of two instances.

- Target SLA: 99.9% uptime (approximately 8.7 hours planned downtime per year).

9 Public Safety and Emergency Handling

When an anomaly score exceeds the detection threshold ($s > 0.75$), the following automated sequence executes within 30 seconds:

1. The RSU logs the incident with GPS coordinates, timestamp, and camera snapshot.
2. An SNS notification is dispatched to the nearest police station and hospital.
3. The cloud route optimiser computes a green corridor: all signals along the fastest path to the incident are switched to a dedicated emergency phase.
4. The dashboard highlights the incident zone and the suggested access route for all traffic control operators simultaneously.
5. After incident clearance (density returns to baseline for three consecutive 30-second windows), signals revert to adaptive mode automatically.

Projected emergency response time with this system: under 9 minutes, satisfying the WHO guideline and reducing current average response times by more than 50%.

10 AWS Cost Estimation

Monthly Operational Cost Breakdown

The following estimates are derived from the AWS Pricing Calculator for the `ap-south-1` region, assuming a pilot deployment covering two metropolitan zones (approximately 50 intersections) with continuous 24/7 operation.

Table 2: Monthly AWS operational cost estimate — pilot deployment (USD).

Service	Configuration	Unit Cost	Monthly (USD)
EC2 (Kinesis consumer)	4× `c6i.2xlarge`, On-Demand	$0.384/hr	$1,105
EC2 (API backend)	3× `t3.medium`, Reserved 1-yr	$0.052/hr	$114
SageMaker Training	2× `ml.p3.2xlarge`, 40 hr/mo	$3.825/hr	$306
SageMaker Inference	4× `ml.m5.xlarge`, On-Demand	$0.269/hr	$776
Kinesis Data Streams	20 shards, 730 hr/mo	$0.015/shard-hr	$219
AWS S3 (Standard)	10 TB/mo + requests	$0.023/GB	$236
S3 Glacier	50 TB archive	$0.004/GB	$205
DynamoDB (on-demand)	50M reads, 20M writes/mo	Variable	$148
Amazon Timestream	500 GB ingest, 60-day retention	$0.50/GB	$310
AWS Lambda	10M invocations/mo	$0.20/1M	$2
Amazon SNS	5M notifications/mo	$0.50/1M	$3
AWS IoT Core	50M messages/mo	$1.00/1M	$50
Data Transfer (out)	2 TB/mo to internet	$0.09/GB	$184
CloudFront CDN	5 TB/mo served	$0.08/GB	$410
Amazon QuickSight	10 authors	$18.00/user	$180
Total			**$4,248**

Cost Distribution

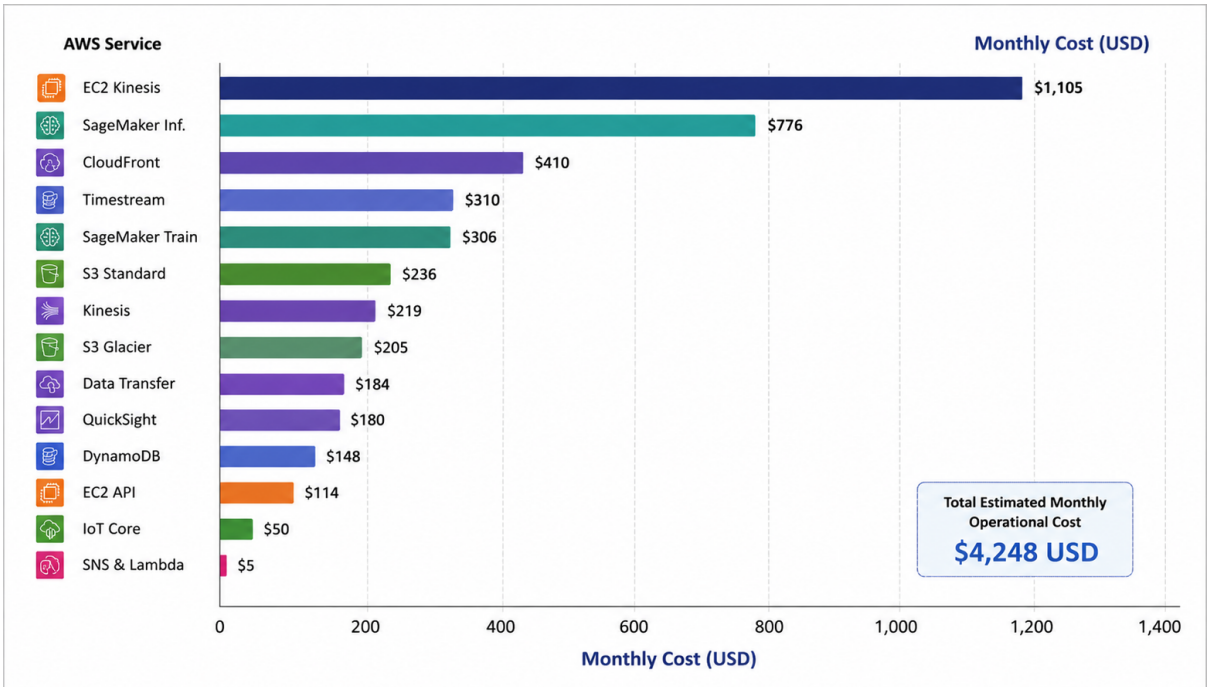

Figure 4: Monthly AWS cost breakdown by service for the pilot deployment across two metropolitan zones. The chart highlights the operational contribution of each AWS service and the total estimated monthly infrastructure cost of $4,248 USD.

Cost Optimisation Strategies

- **Reserved Instances:** Converting stable EC2 On-Demand workloads to 1-year Reserved pricing reduces compute costs by 30–40%.
- **S3 Lifecycle Policies:** Automatic tiering of data older than 30 days to Glacier reduces storage cost per GB from $0.023 to $0.004.
- **Edge Inference:** Running signal optimisation locally on RSUs reduces SageMaker inference calls by an estimated 70%.
- **Spot Instances for Training:** Using EC2 Spot for SageMaker training jobs reduces training costs by up to 70%.
- Full-city deployment at 600+ intersections reduces cost per intersection from approximately $85/month (pilot) to $38/month at scale.

11 Analytical Discussion: Conflicting Technical Requirements

Low Latency vs. Centralised Processing

Signal timing decisions require sub-200 ms response, incompatible with cloud round-trips (typical RTT: 80–150 ms plus processing overhead). The hybrid design resolves this by

delegating time-critical inference to RSUs while the cloud handles model retraining and long-horizon forecasting. The trade-off is increased edge hardware cost and operational complexity.

Data Sharing vs. User Privacy

Vehicle tracking data, combined with GPS and camera feeds, constitutes personally identifiable information. Mitigations include: (a) facial blurring at the camera before any frame leaves the RSU; (b) licence plate hashing in DynamoDB records; and (c) intersection-level aggregation before cloud upload, so individual trajectories are never transmitted. The residual conflict is that emergency routing benefits from individual vehicle positions, which privacy constraints restrict. Federated GPS aggregation is considered for a future phase.

High Accuracy vs. Computational Cost

LSTM models achieve the highest forecasting accuracy but are too expensive for on-device inference. The system uses a two-tier strategy: a lightweight XGBoost model on each RSU for real-time signal control, and a full LSTM on a SageMaker endpoint for 30-minute ahead city-level forecasting. This decouples accuracy requirements from latency requirements.

System Interdependencies

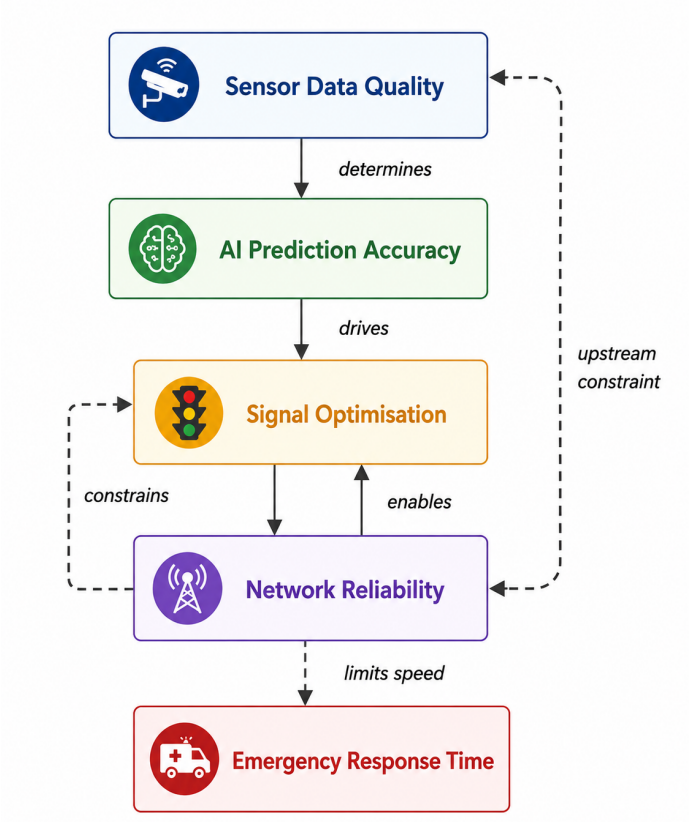

Figure 5: Vertical interdependency map of key system subproblems. Solid arrows indicate functional dependencies, while Dashed arrows represent constraint relationships between different system components.

12 Adherence to Standards

Table 3: Standards and protocols adopted in the proposed system.

Domain	**Standard / Protocol**	**Application in System**
IoT Communication	MQTT 3.1.1 / MQTT 5.0	Device-to-edge telemetry
Transport Security	TLS 1.3	All network channels
Data Encryption	AES-256 (SSE-KMS)	S3 and DynamoDB at rest
Device Identity	X.509 certificates	RSU authentication to AWS IoT
ITS Interface	ISO 14906	Electronic fee collection interface
Video Surveillance	ONVIF Profile S	IP camera interoperability
API Design	REST / OpenAPI 3.0	Dashboard and third-party APIs
Access Control	AWS IAM (RBAC)	All cloud service access
Incident Alerting	CAP (Common Alerting Protocol)	Emergency broadcast messaging

13 Stakeholder Analysis

Table 4: Stakeholder requirements and how the system addresses them.

Stakeholder	**Primary Concern**	**System Response**
Government / City Corp.	Infrastructure ROI, regulatory compliance	AWS cost dashboard, compliance-by-design
Commuters	Reduced travel time, predictable journeys	Real-time mobile app with ETA updates
Traffic Police	Reduced manual workload, instant alerts	Automated signal control, SNS alerts
Emergency Services	Fastest route, pre-cleared signal corridor	Automated emergency phase and routing
BTRC / ICT Division	Data sovereignty, radio spectrum use	On-premise RSU option, licensed LTE

14 System Performance Projections

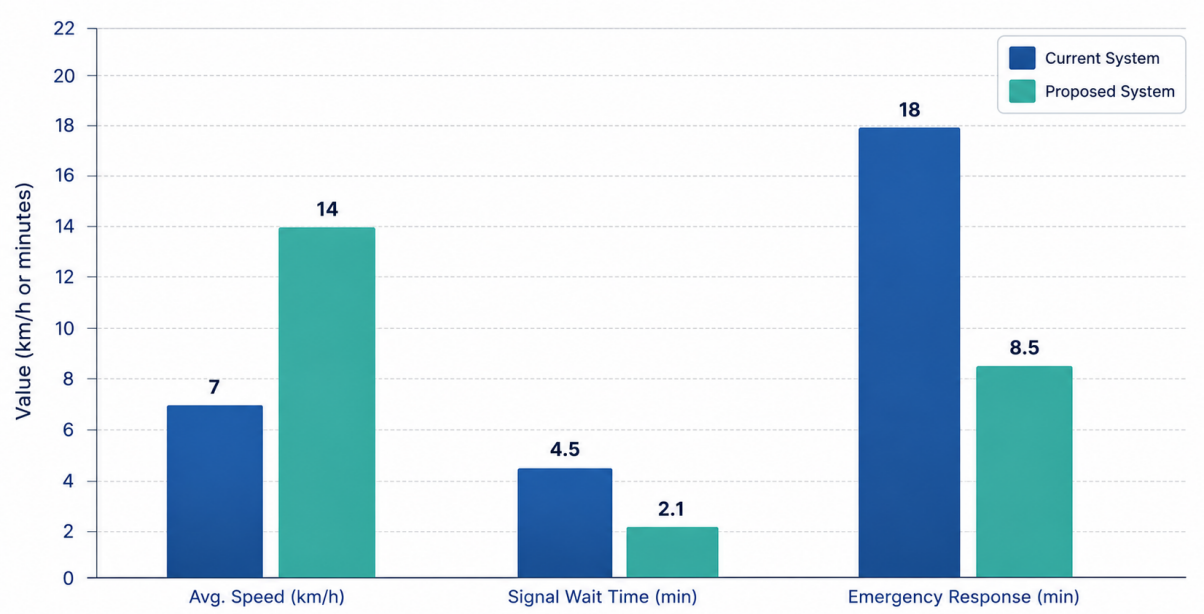

Figure 6: Key performance metric comparison between the current traffic management system and the proposed AI-based adaptive system. Incident detection time (not charted) improves from manual detection taking hours to automated detection in under 30 seconds.

15 Conclusion

The proposed Intelligent Smart Transportation and Traffic Management System (ISTTMS) provides a practical and scalable solution to Bangladesh's urban traffic challenges. By integrating IoT sensors, edge computing, AI-based analytics, and AWS cloud services, the system enables intelligent and real-time traffic management.

The platform supports traffic monitoring through smart cameras, GPS-enabled vehicles, traffic signals, and environmental sensors. AI-driven adaptive signal control helps reduce congestion by optimizing traffic flow based on real-time conditions. The system also includes automated emergency response features that detect incidents quickly and create green signal corridors for emergency vehicles.

A hybrid AWS architecture ensures scalability, reliability, and multi-city deployment support. Cost efficiency is achieved through edge-cloud workload distribution, Reserved Instances, Spot Instances, and optimized cloud storage management.

Overall, the system is secure, cost-effective, and suitable for Bangladesh's existing infrastructure, providing a strong foundation for future smart city transportation development.